

Chapter 12 — Extension

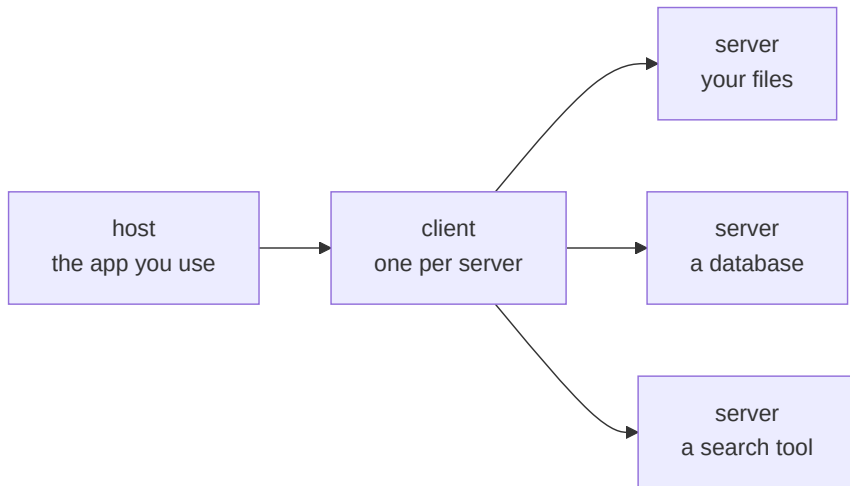
Reaching outside the session

Chapter 6 drew a trust boundary. This chapter is the room opening it on purpose, knowing what it costs.

The problem a protocol solves

- Every tool you might connect — a database, a document store, a second provider — has its own interface.
- Without a common protocol that is N tools times M clients of bespoke integration, and none of it is reusable.
- A protocol collapses that to N plus M . That is the entire motivation, and it is the same argument as any other interface standard you have met.

Host, client, server



- The **server** exposes capability. It is usually small and often something you can write.
- The **client** speaks the protocol. You do not write it.
- The **host** is the application you are already using.

Three things a server can expose

- **Tools** — actions the model may call. This is the one with consequences.
- **Resources** — data the model may read. Files, records, documents.
- **Prompts** — reusable prompt templates the server offers to the host.
- The spec also lets *clients* offer features back to servers — sampling, roots, elicitation. Out of scope here; know they exist.

Keep the first two distinct in your head. A tool **does** something; a resource is something read. Chapter 6's whole failure mode is content from the second class being treated as an instruction from the first.

The protocol says this itself. Tools "represent arbitrary code execution and must be treated with appropriate caution", and tool descriptions "should be considered untrusted, unless obtained from a trusted server." The threat model is in the specification, not bolted on by this course.

Graphing a codebase or a document set

- Build a local, queryable map of code, documents and PDFs, and then ask questions of the map instead of reading files one at a time.
- Edges are labelled by how they were obtained: **extracted** from the syntax tree, or **inferred**. Those are not the same evidence and the map says which.
- Commit the map so the group shares one picture rather than six.

Two cautions that belong together. Pin the version of a fast-moving dependency, or the map you commit today is not the map you rebuild next month. And the **semantic pass over documents leaves the machine while the code pass does not** — which makes it a Chapter 18 data-governance question, not a convenience question.

That split was confirmed by running it: on version 0.9.16 the document pass refuses without an external API key, and the code pass runs on a local syntax tree without one. It is a **version-pinned observation**, not a documented guarantee — re-check it on upgrade, because Chapter 18 routes an NDA decision through it.

Cross-provider retrieval: a licensing asymmetry

Different providers hold different content licences. A second provider reachable from the same session can retrieve sources the first is not licensed to return.

- Taught as a **licensing asymmetry with a provenance obligation**. Not as circumvention, and not as a trick.
- **Onward use may exceed the retrieving provider's terms**. Retrieval is not a licence to republish.
- **Source quality still has to be judged**. A second provider is not a second opinion.
- **Record the retrieval chain**. Which provider returned what, and when. Chapter 14 will require the identifier anyway.

Framed as circumvention this would be both wrong and fragile. Framed as an asymmetry with an obligation attached, it survives a question from a librarian.

Trust boundary, reopened

- Chapter 6: untrusted content entering the window is treated as instruction, and there is no complete defence.
- This chapter: you are now connecting servers that fetch content you did not write, on purpose, because it is useful.
- Nothing about the threat changed. What changed is that you chose it.

Thread — trust boundary. Introduced Chapter 2 as trained behaviour, attacked in Chapter 6, opened voluntarily here, and governed in Chapter 18.

Hands-on

BLOCKED — needs the working Antigravity configuration.

This chapter cannot be finished until the group's Gemini access path is documented: what is actually invoked, and how results come back. See `DECISIONS.md` item 2.

The preceding slides state the **mechanism**, which stands. But the cross-provider slide's central factual claim — that a second provider is reachable from this session and returns sources the first cannot — is **exactly what the missing configuration would establish**, so it is unverified until then. The hands-on exercise depends on it entirely.

The exercise, once unblocked: connect one server, list what it exposes, call one tool, read one resource, and record where the data went.

Where this leaves us

- A protocol, so a tool written once is reachable from anything that speaks it.
- Three server primitives, and the distinction between the one that **acts** and the one that is **read** — which is Chapter 6's failure mode restated as an interface.
- A map of your own code and documents, with the local and the remote halves told apart.
- A retrieval asymmetry you may use, with an obligation attached rather than a trick.

Every channel in this chapter was opened deliberately. **Chapter 13** is what it costs to keep them open — and Chapter 18 is who answers for what went through them.